

DOMAIN 2

CLAUDE CODE CONFIGURATION & WORKFLOWS

Complete deep-dive for this blueprint domain (20% of exam) of the Claude Certified Architect — Foundations exam: every concept with worked examples, a dedicated traps-and-nuances section explaining why the tempting wrong answers are wrong, a one-page cheat sheet, and a full bank of 120 original practice questions with answers and rationales.

Section	What's inside
1. Cheat Sheet	Golden rule, quick-reference tables, and a scenario decision checklist — read this first and last.
2. Concept Guide	Full narrative coverage of every concept tested in this domain, with worked scenarios.
3. Traps & Nuances	16 recurring wrong-answer patterns: the trap, why it's tempting, and the correct principle.
4. Practice Questions	120 original scenario-based questions, 4 options each, answer marked, rationale included.

This is an independent study aid, not an official Anthropic publication or a reproduction of actual exam questions, which Anthropic does not publish.

1 · CHEAT SHEET

Golden rule: shared, durable content (team conventions, shared MCP servers) goes in version-controlled project/directory files; personal, experimental, or local-only content stays out of them. Almost every Domain 2 trap is a scope-placement mistake.

CLAUDE.md hierarchy

<code>~/.claude/CLAUDE.md</code>	User-level. Personal. NOT shared, NOT version-controlled.
<code>.claude/CLAUDE.md</code>	Project-level. Shared team conventions. Version-controlled.
Directory CLAUDE.md	Scoped to one subfolder/package. Shared, version-controlled.

MCP config scope

<code>.mcp.json</code>	Project-level, shared, version-controlled MCP servers.
<code>~/.claude.json</code>	Personal / experimental MCP servers, not shared.
Secrets	Always <code>\${ENV_VAR}</code> substitution — never a literal committed value.

Hooks & isolation

PreToolUse hook	Runs BEFORE a call — use to block/gate.
PostToolUse hook	Runs AFTER a call — use to trim/redact/log results.
context: fork (Skill)	Runs the skill in an isolated subagent — shields main context from verbose output.
<code>.claude/rules/</code> paths:	YAML frontmatter glob — loads a rule only for matching files.

CI / automation essentials

-p (print) flag	Required for non-interactive/CI use. Without it, sessions hang waiting on stdin.
Independent review	Use a fresh session with no generation context — the authoring session rationalizes its own work.

Decision checklist for any Domain 2 scenario

- Convention followed by only one person / not propagating? → it's almost certainly in a user-level file, not project-level.
- CI job hangs with no output? → missing `-p`.
- Want a rule to load only for certain files? → add `paths: frontmatter` in `.claude/rules/`.
- Skill produces huge verbose output? → `context: fork`.
- Server used by the whole team vs. just you? → `.mcp.json` vs. `~/.claude.json`.
- Any credential in a config file? → must be `${ENV_VAR}`, never literal.
- Reviewing sensitive/large code? → use an independent session, not the authoring one.

Claude Code Configuration & Workflows

2 · CONCEPT GUIDE

This domain tests whether you can configure Claude Code for team-scale, CI-safe, and context-efficient use — not whether you can write a single prompt. Expect scenario questions about misconfigured hierarchy, context bloat, and non-interactive automation.

2.1 The CLAUDE.md hierarchy

Claude Code resolves project context from three levels, each with a different scope and lifecycle:

Level	Path	Scope	Version-controlled?
User	~/.claude/CLAUDE.md	Personal preferences, applies across all your projects	No — local to you
Project	.claude/CLAUDE.md	Team conventions, shared across everyone working on the repo	Yes — committed
Directory	CLAUDE.md in a subfolder	Conventions specific to that part of the codebase	Yes — committed

Worked scenario

A new developer joins a team and consistently fails to follow the project's established coding conventions, even though “the conventions are documented.” Most likely cause: whoever wrote them added the conventions to their own **user-level** CLAUDE.md rather than the **project-level** file — so the rules only ever applied to that one person's sessions and were never shared with the team. Fix: move team conventions into **.claude/CLAUDE.md** and commit it.

2.2 Conditional rules with .claude/rules/

Rules in **.claude/rules/** use YAML frontmatter to load conditionally based on file path glob patterns, keeping context lean by only injecting conventions relevant to the files actually being touched:

```
---
paths: ["**/*.test.tsx"]
---
# Testing conventions – only loaded when working on test files
Use React Testing Library, not Enzyme. Prefer user-event over fireEvent.

---
paths: ["src/api/**/*.ts"]
---
# API conventions – only loaded when working on API files
All endpoints must validate input with zod before touching the DB layer.
```

The exam payoff: a rule scoped with **paths: ["**/*.test.tsx"]** never loads while you're editing an API route, and vice versa — irrelevant conventions don't consume context on every operation.

2.3 Skills and context isolation with context: fork

A Skill declared with **context: fork** runs in an isolated subagent rather than the main session. This matters for any skill that produces verbose intermediate output — a dependency scan, a full codebase map, a large coverage report. That

output stays contained inside the forked subagent's context and only a distilled result returns to the main session, protecting the primary conversation's context budget.

2.4 Non-interactive / CI usage

The **-p** (print) flag runs Claude Code non-interactively: it processes the prompt, writes the result to stdout, and exits — the correct mode for CI/CD pipelines, cron jobs, and scripted automation. Without it, Claude Code launches its interactive REPL and a pipeline step will simply hang waiting for terminal input that never arrives.

```
claude -p "Summarize failing tests in this branch and suggest fixes" > report.txt
```

Exam trap: distractor options often include invented flags or hacks that sound plausible — a nonexistent environment variable, redirecting stdin from /dev/null as a workaround, or a nonexistent **--batch** flag. If you've actually run Claude Code in a pipeline, **-p** is a free point; if you haven't, memorize it explicitly.

2.5 MCP configuration scope

- **.mcp.json** — project-level MCP server configuration, version-controlled and shared across the whole team.
- **~/.claude.json** — personal/experimental MCP servers, not shared, useful for servers you're trialing before proposing them for the team config.
- Secrets in either file should use **environment-variable substitution** (e.g. **\${GITHUB_TOKEN}**) rather than committed literal values — committing a real token into **.mcp.json** is a textbook exam distractor to reject.

2.6 Independent review & session isolation

A model instance that just generated a piece of code carries the context and reasoning that led to that code — when asked to review its own output in the same session, it tends to rationalize its earlier decisions rather than catch them. An independent session or subagent with no generation context, asked purely to review, catches more issues. This is the Claude Code analog of “don't grade your own homework,” and the exam tests it as a reason to deliberately spin up a fresh review session rather than continuing in the same thread.

2.7 Quick-reference command & config table

Item	Purpose
-p / --print	Non-interactive mode for CI/CD and scripting
.claude/CLAUDE.md	Shared, version-controlled project conventions
~/.claude/CLAUDE.md	Personal, local-only preferences
.claude/rules/*.md (with paths: frontmatter)	Conditionally-loaded, path-scoped conventions
context: fork (in a Skill)	Runs the skill in an isolated subagent to shield main context
.mcp.json	Shared, versioned MCP server config
~/.claude.json	Personal/experimental MCP server config
\${ENV_VAR} in MCP config	Secret injection without committing literal credentials

3 · TRAPS & NUANCES

Each card below names a specific concept, describes the wrong answer the exam is likely to dangle in front of you, explains why that wrong answer feels right, and states the principle that actually resolves it. Read this section right before the exam as a final refresh.

CONCEPT: CLAUDE.md hierarchy

The trap: Assuming any CLAUDE.md file, wherever it lives, is automatically shared with the team.

Why it's tempting: It's "just a config file," so it's easy to assume all instances behave the same way.

The correct principle: Only project- and directory-level CLAUDE.md files are version-controlled and shared; ~/.claude/CLAUDE.md is personal and never propagates to teammates.

CONCEPT: -p / non-interactive mode

The trap: Assuming Claude Code will work fine in a CI pipeline with no special flags, since it worked fine in a local terminal.

Why it's tempting: Local terminal usage never required any special flag, so it's easy to assume the default behavior transfers.

The correct principle: CI runners provide no interactive input; without -p, Claude Code launches its interactive mode and the job hangs indefinitely.

CONCEPT: .claude/rules/ path scoping

The trap: Assuming a rule file placed inside .claude/rules/ is automatically conditional, even with no paths frontmatter.

Why it's tempting: The folder name "rules" implies selective, conditional application.

The correct principle: Conditional loading requires explicit paths frontmatter — a rule with none can load unconditionally, defeating the purpose of scoping.

CONCEPT: Skill context isolation

The trap: Assuming any Skill automatically runs in isolated context by default.

Why it's tempting: "Skill" sounds modular and self-contained, implying isolation is built in.

The correct principle: Context isolation requires explicitly setting context: fork on the Skill — without it, verbose output runs straight into the main session's context.

CONCEPT: MCP config scope for experiments

The trap: Adding an experimental or personal MCP server to .mcp.json "just to try it out," since that's the main config file.

Why it's tempting: .mcp.json is the more prominent, "official-looking" config location.

The correct principle: Experimental or personal servers belong in ~/.claude.json until they're ready to be proposed for the shared, version-controlled project config.

CONCEPT: Secrets handling

The trap: Base64-encoding a secret before committing it and treating that as equivalent to not exposing it.

Why it's tempting: Encoding changes how the secret looks, which can feel like a form of protection.

The correct principle: Encoding is not secrecy — it's trivially reversible. Secrets must flow through environment-variable substitution, never as any form of committed literal value.

CONCEPT: Session review independence

The trap: Trusting a thorough self-review from the same Claude Code session that just wrote the code.

Why it's tempting: That session has full context on the change and “just worked on it,” so it seems well-positioned to review it.

The correct principle: A session that generated code tends to rationalize its own earlier decisions; an independent session with no generation context catches more issues.

CONCEPT: PreToolUse vs. PostToolUse hooks

The trap: Using a PostToolUse hook to try to block a disallowed tool call.

Why it's tempting: “Hook” sounds like a generic interception point, so it's easy to assume either type can block execution.

The correct principle: PostToolUse only runs after the call has already executed — blocking or gating requires a PreToolUse hook, which fires before execution.

CONCEPT: Tool permission scoping

The trap: Granting a session broad, unscoped tool permissions “to avoid friction” during setup.

Why it's tempting: Broad permissions reduce the chance of hitting a blocked action mid-task, so it feels like it saves time.

The correct principle: Least-privilege scoping — matching permissions to what the specific task actually needs — limits the damage if a session goes off track, even at the cost of some setup friction.

CONCEPT: Monorepo CLAUDE.md structure

The trap: Cramming every package's conventions into a single top-level project CLAUDE.md “to keep it simple.”

Why it's tempting: One file feels easier to maintain and find than several scattered ones.

The correct principle: Directory-level CLAUDE.md files let each package carry only its own relevant conventions, keeping unrelated rules out of context when working elsewhere in the repo.

CONCEPT: Diagnosing context bloat

The trap: Reaching straight for a larger context-window model when a session's context usage climbs unexpectedly.

Why it's tempting: A bigger context window is a simple, direct-feeling fix for “running out of room.”

The correct principle: The first step should be inspecting what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) and trimming it — a bigger window just delays the same underlying inefficiency.

CONCEPT: Repeatable workflows

The trap: Re-typing the same multi-step prompt from memory each time a routine task comes up, since it “isn't that long.”

Why it's tempting: Setting up a custom command feels like more upfront work than just retyping a familiar prompt.

The correct principle: Repeatable, well-defined workflows are better encoded as custom slash commands — consistent, versionable, and immune to the drift that comes from retyping from memory.

CONCEPT: Onboarding new contributors

The trap: Relying on shadowing a senior engineer or informal tribal knowledge instead of written, shared configuration.

Why it's tempting: Personal mentorship feels more thorough and adaptable than a static file.

The correct principle: The durable fix is putting conventions in version-controlled project- or directory-level CLAUDE.md — tribal knowledge doesn't scale and doesn't survive team turnover.

CONCEPT: Enforcing testing conventions

The trap: Relying on code review to catch convention violations after the fact, since review is already part of the process.

Why it's tempting: Review is a known, existing checkpoint, so it feels like it should be sufficient.

The correct principle: Path-scoped rules prevent violations before they happen by loading the relevant convention automatically whenever matching files are touched — catching things at review time is strictly later and less reliable.

CONCEPT: Extended thinking budget

The trap: Always maxing out the reasoning/thinking budget “to be safe,” regardless of task difficulty.

Why it's tempting: More reasoning intuitively feels like it can only help, never hurt.

The correct principle: Reasoning budget should scale with actual task difficulty — over-thinking trivial tasks wastes tokens and latency without improving the outcome.

CONCEPT: Output style selection

The trap: Using one verbose, explanatory output style for both machine-readable and human-facing consumers.

Why it's tempting: Verbose output feels more “complete” and safer by default.

The correct principle: Output style should match the consumer — a machine-readable pipeline needs minimal, parseable output; a human reader benefits from explanatory framing.

4 · PRACTICE QUESTIONS (120)

Q1. Where should a personal shortcut for how you like commit messages phrased, used only in your own sessions be documented for Claude Code?

- A. `.claude/CLAUDE.md` (project-level)
- B. A directory-level `CLAUDE.md` inside that module
- C. The system prompt, hardcoded
- D. ✓ `~/.claude/CLAUDE.md` (user-level)

Correct answer: D. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q2. Where should the team's shared testing conventions, meant to apply to every contributor be documented for Claude Code?

- A. A directory-level `CLAUDE.md` inside that module
- B. The system prompt, hardcoded
- C. ✓ `.claude/CLAUDE.md` (project-level)
- D. `~/.claude/CLAUDE.md` (user-level)

Correct answer: C. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q3. Where should conventions specific only to the payments module, not the rest of the codebase be documented for Claude Code?

- A. The system prompt, hardcoded
- B. ✓ A directory-level `CLAUDE.md` inside that module
- C. `~/.claude/CLAUDE.md` (user-level)
- D. `.claude/CLAUDE.md` (project-level)

Correct answer: B. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q4. Where should your own personal preference for verbose vs. terse explanations be documented for Claude Code?

- A. ✓ `~/.claude/CLAUDE.md` (user-level)
- B. `.claude/CLAUDE.md` (project-level)
- C. A directory-level `CLAUDE.md` inside that module
- D. The system prompt, hardcoded

Correct answer: A. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q5. Where should the repo-wide required code style guide, committed and shared be documented for Claude Code?

- A. `~/.claude/CLAUDE.md` (user-level)
- B. A directory-level `CLAUDE.md` inside that module
- C. The system prompt, hardcoded
- D. ✓ `.claude/CLAUDE.md` (project-level)

Correct answer: D. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q6. Where should naming conventions that only apply inside the /frontend directory be documented for Claude Code?

- A. .claude/CLAUDE.md (project-level)
- B. A directory-level CLAUDE.md inside that module
- C. ✓ A directory-level CLAUDE.md inside /frontend
- D. ~/.claude/CLAUDE.md (user-level)

Correct answer: C. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q7. Where should a note reminding yourself to always double-check SQL migrations, just for your sessions be documented for Claude Code?

- A. The system prompt, hardcoded
- B. ✓ ~/.claude/CLAUDE.md (user-level)
- C. .claude/CLAUDE.md (project-level)
- D. A directory-level CLAUDE.md inside that module

Correct answer: B. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q8. Where should the team's shared API design conventions everyone must follow be documented for Claude Code?

- A. ✓ .claude/CLAUDE.md (project-level)
- B. ~/.claude/CLAUDE.md (user-level)
- C. A directory-level CLAUDE.md inside that module
- D. The system prompt, hardcoded

Correct answer: A. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q9. Where should database schema conventions specific to the /db directory only be documented for Claude Code?

- A. ~/.claude/CLAUDE.md (user-level)
- B. .claude/CLAUDE.md (project-level)
- C. A directory-level CLAUDE.md inside that module
- D. ✓ A directory-level CLAUDE.md inside /db

Correct answer: D. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q10. Where should your personal preference for how much extended thinking to show, local only to you be documented for Claude Code?

- A. A directory-level CLAUDE.md inside that module
- B. The system prompt, hardcoded
- C. ✓ ~/.claude/CLAUDE.md (user-level)
- D. .claude/CLAUDE.md (project-level)

Correct answer: C. CLAUDE.md resolves at three levels: user (personal, not shared), project (shared, version-controlled, repo-wide), and directory (shared, scoped to a subfolder) — match the scope of the content to the level.

Q11. On a team using Claude Code, a new developer never follows the documented commit-message format, even though the convention is "documented." What's the likely cause?

- A. CLAUDE.md files are ignored unless referenced in every prompt manually.
- B. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- C. Claude Code doesn't support enforcing coding conventions at all.
- D. The convention must have been written in a language Claude doesn't understand.

Correct answer: B. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q12. On a team using Claude Code, only one senior engineer's sessions seem to respect the linting rules, even though the convention is "documented." What's the likely cause?

- A. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- B. Claude Code doesn't support enforcing coding conventions at all.
- C. The convention must have been written in a language Claude doesn't understand.
- D. CLAUDE.md files are ignored unless referenced in every prompt manually.

Correct answer: A. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q13. On a team using Claude Code, the team's API error-handling convention is followed inconsistently across contributors, even though the convention is "documented." What's the likely cause?

- A. Claude Code doesn't support enforcing coding conventions at all.
- B. The convention must have been written in a language Claude doesn't understand.
- C. CLAUDE.md files are ignored unless referenced in every prompt manually.
- D. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.

Correct answer: D. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q14. On a team using Claude Code, a documented testing convention only ever applies when the original author is working, even though the convention is "documented." What's the likely cause?

- A. The convention must have been written in a language Claude doesn't understand.
- B. CLAUDE.md files are ignored unless referenced in every prompt manually.
- C. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- D. Claude Code doesn't support enforcing coding conventions at all.

Correct answer: C. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q15. On a team using Claude Code, new hires consistently miss the team's naming conventions despite "documentation" existing, even though the convention is "documented." What's the likely cause?

- A. CLAUDE.md files are ignored unless referenced in every prompt manually.
- B. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- C. Claude Code doesn't support enforcing coding conventions at all.
- D. The convention must have been written in a language Claude doesn't understand.

Correct answer: B. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q16. On a team using Claude Code, a security review checklist is followed by only one contributor's sessions, even though the convention is "documented." What's the likely cause?

- A. ✓ The convention was likely placed in a user-level CLAUDE.md rather than the project-level file, so it never propagated to the team.
- B. Claude Code doesn't support enforcing coding conventions at all.
- C. The convention must have been written in a language Claude doesn't understand.
- D. CLAUDE.md files are ignored unless referenced in every prompt manually.

Correct answer: A. This is the textbook CLAUDE.md hierarchy trap: content placed in ~/.claude/CLAUDE.md is personal and never shared, so it only ever applies to the one person who wrote it.

Q17. You want a rule in .claude/rules/ to load only when editing test files, and stay out of context otherwise. How do you scope it?

- A. Rename the rule file to match the folder name exactly.
- B. Put the rule inside the project-level CLAUDE.md instead.
- C. Duplicate the rule into every relevant subdirectory manually.
- D. ✓ Add YAML frontmatter to the rule file with paths: ["**/*.test.tsx"]

Correct answer: D. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q18. You want a rule in .claude/rules/ to load only when editing files under the API layer, and stay out of context otherwise. How do you scope it?

- A. Put the rule inside the project-level CLAUDE.md instead.
- B. Duplicate the rule into every relevant subdirectory manually.
- C. ✓ Add YAML frontmatter to the rule file with paths: ["src/api/**/*.ts"]
- D. Rename the rule file to match the folder name exactly.

Correct answer: C. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q19. You want a rule in .claude/rules/ to load only when editing database migration files, and stay out of context otherwise. How do you scope it?

- A. Duplicate the rule into every relevant subdirectory manually.
- B. ✓ Add YAML frontmatter to the rule file with paths: ["**/migrations/*.sql"]
- C. Rename the rule file to match the folder name exactly.
- D. Put the rule inside the project-level CLAUDE.md instead.

Correct answer: B. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q20. You want a rule in .claude/rules/ to load only when editing files under the frontend directory, and stay out of context otherwise. How do you scope it?

- A. ✓ Add YAML frontmatter to the rule file with paths: ["src/frontend/**/*.ts"]
- B. Rename the rule file to match the folder name exactly.
- C. Put the rule inside the project-level CLAUDE.md instead.
- D. Duplicate the rule into every relevant subdirectory manually.

Correct answer: A. Rules in .claude/rules/ load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q21. You want a rule in `.claude/rules/` to load only when editing configuration YAML files, and stay out of context otherwise. How do you scope it?

- A. Rename the rule file to match the folder name exactly.
- B. Put the rule inside the project-level `CLAUDE.md` instead.
- C. Duplicate the rule into every relevant subdirectory manually.
- D. ✓ Add YAML frontmatter to the rule file with paths: `["**/*.yaml", "**/*.yml"]`

Correct answer: D. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q22. You want a rule in `.claude/rules/` to load only when editing Python files under the ML pipeline directory, and stay out of context otherwise. How do you scope it?

- A. Put the rule inside the project-level `CLAUDE.md` instead.
- B. Duplicate the rule into every relevant subdirectory manually.
- C. ✓ Add YAML frontmatter to the rule file with paths: `["src/ml/**/*.py"]`
- D. Rename the rule file to match the folder name exactly.

Correct answer: C. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q23. You want a rule in `.claude/rules/` to load only when editing documentation markdown files, and stay out of context otherwise. How do you scope it?

- A. Duplicate the rule into every relevant subdirectory manually.
- B. ✓ Add YAML frontmatter to the rule file with paths: `["docs/**/*.md"]`
- C. Rename the rule file to match the folder name exactly.
- D. Put the rule inside the project-level `CLAUDE.md` instead.

Correct answer: B. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q24. You want a rule in `.claude/rules/` to load only when editing infrastructure-as-code files, and stay out of context otherwise. How do you scope it?

- A. ✓ Add YAML frontmatter to the rule file with paths: `["infra/**/*.tf"]`
- B. Rename the rule file to match the folder name exactly.
- C. Put the rule inside the project-level `CLAUDE.md` instead.
- D. Duplicate the rule into every relevant subdirectory manually.

Correct answer: A. Rules in `.claude/rules/` load conditionally based on YAML frontmatter path globs — this keeps conventions out of context except when working on files they actually apply to.

Q25. A Skill performs a full dependency-vulnerability scan across the repo, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Run the skill directly in the main session for simplicity.
- B. Split the skill's output across multiple `CLAUDE.md` files.
- C. Increase the main session's context window instead.
- D. ✓ Configure the Skill with `context: fork` so it runs in an isolated subagent.

Correct answer: D. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q26. A Skill performs a deep codebase map generation step, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Split the skill's output across multiple `CLAUDE.md` files.
- B. Increase the main session's context window instead.
- C. ✓ Configure the Skill with `context: fork` so it runs in an isolated subagent.
- D. Run the skill directly in the main session for simplicity.

Correct answer: C. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q27. A Skill performs a large test-coverage report, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Increase the main session's context window instead.
- B. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.
- C. Run the skill directly in the main session for simplicity.
- D. Split the skill's output across multiple CLAUDE.md files.

Correct answer: B. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q28. A Skill performs an exhaustive static-analysis pass, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.
- B. Run the skill directly in the main session for simplicity.
- C. Split the skill's output across multiple CLAUDE.md files.
- D. Increase the main session's context window instead.

Correct answer: A. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q29. A Skill performs a bulk log-file anomaly scan, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Run the skill directly in the main session for simplicity.
- B. Split the skill's output across multiple CLAUDE.md files.
- C. Increase the main session's context window instead.
- D. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.

Correct answer: D. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q30. A Skill performs a large schema-diff report between two databases, producing very verbose output. How do you keep that output from bloating the main session's context?

- A. Split the skill's output across multiple CLAUDE.md files.
- B. Increase the main session's context window instead.
- C. ✓ Configure the Skill with context: fork so it runs in an isolated subagent.
- D. Run the skill directly in the main session for simplicity.

Correct answer: C. context: fork runs a Skill in an isolated subagent; only a distilled result returns to the main session, containing verbose intermediate output outside the primary conversation's context budget.

Q31. You're setting up a GitHub Actions job that runs Claude Code to summarize failing tests and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Set a made-up CLAUDE_HEADLESS=true environment variable.
- B. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- C. Run Claude Code with no special flags; the default mode works fine in CI.
- D. Redirect stdin from /dev/null as the primary fix.

Correct answer: B. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q32. You're setting up a nightly cron job that asks Claude Code to generate a changelog and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- B. Run Claude Code with no special flags; the default mode works fine in CI.
- C. Redirect stdin from /dev/null as the primary fix.
- D. Set a made-up CLAUDE_HEADLESS=true environment variable.

Correct answer: A. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q33. You're setting up a pre-commit hook that runs Claude Code to check for TODOs and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Run Claude Code with no special flags; the default mode works fine in CI.
- B. Redirect stdin from /dev/null as the primary fix.
- C. Set a made-up CLAUDE_HEADLESS=true environment variable.
- D. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.

Correct answer: D. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q34. You're setting up a Jenkins pipeline stage that runs Claude Code to lint recent changes and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Redirect stdin from /dev/null as the primary fix.
- B. Set a made-up CLAUDE_HEADLESS=true environment variable.
- C. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- D. Run Claude Code with no special flags; the default mode works fine in CI.

Correct answer: C. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q35. You're setting up a scheduled job that has Claude Code generate a dependency report and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Set a made-up CLAUDE_HEADLESS=true environment variable.
- B. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- C. Run Claude Code with no special flags; the default mode works fine in CI.
- D. Redirect stdin from /dev/null as the primary fix.

Correct answer: B. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q36. You're setting up a CI stage that has Claude Code draft release notes and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- B. Run Claude Code with no special flags; the default mode works fine in CI.
- C. Redirect stdin from /dev/null as the primary fix.
- D. Set a made-up CLAUDE_HEADLESS=true environment variable.

Correct answer: A. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q37. You're setting up an automated script that has Claude Code review a diff and post a comment and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Run Claude Code with no special flags; the default mode works fine in CI.
- B. Redirect stdin from /dev/null as the primary fix.
- C. Set a made-up CLAUDE_HEADLESS=true environment variable.
- D. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.

Correct answer: D. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q38. You're setting up a batch job that has Claude Code triage new bug reports and the pipeline step hangs indefinitely with no output. What's most likely missing?

- A. Redirect stdin from /dev/null as the primary fix.
- B. Set a made-up CLAUDE_HEADLESS=true environment variable.
- C. ✓ Use the -p (print) flag so Claude Code runs non-interactively and exits after producing output.
- D. Run Claude Code with no special flags; the default mode works fine in CI.

Correct answer: C. Without -p, Claude Code launches its interactive REPL and waits for terminal input that a CI runner never provides, causing the job to hang; -p runs it non-interactively and exits after completion.

Q39. Where should the configuration for a database MCP server the whole team needs for the project live?

- A. An environment variable with no config file
- B. ✓ .mcp.json (project-level, version-controlled)
- C. ~/.claude.json (personal, not shared)
- D. Directly inside CLAUDE.md as free text

Correct answer: B. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q40. Where should the configuration for an experimental MCP server one engineer is trialing before proposing it to the team live?

- A. ✓ ~/.claude.json (personal, not shared)
- B. .mcp.json (project-level, version-controlled)
- C. Directly inside CLAUDE.md as free text
- D. An environment variable with no config file

Correct answer: A. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q41. Where should the configuration for a shared internal ticketing MCP server required for everyone's workflow live?

- A. ~/.claude.json (personal, not shared)
- B. Directly inside CLAUDE.md as free text
- C. An environment variable with no config file
- D. ✓ .mcp.json (project-level, version-controlled)

Correct answer: D. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q42. Where should the configuration for a personal weather MCP server one developer likes to use locally live?

- A. Directly inside CLAUDE.md as free text
- B. An environment variable with no config file
- C. ✓ ~/.claude.json (personal, not shared)
- D. .mcp.json (project-level, version-controlled)

Correct answer: C. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q43. Where should the configuration for a shared internal search MCP server the whole repo depends on live?

- A. An environment variable with no config file
- B. ✓ .mcp.json (project-level, version-controlled)
- C. ~/.claude.json (personal, not shared)
- D. Directly inside CLAUDE.md as free text

Correct answer: B. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q44. Where should the configuration for a personal note-taking MCP server unrelated to the team's actual project live?

- A. ✓ ~/.claude.json (personal, not shared)
- B. .mcp.json (project-level, version-controlled)
- C. Directly inside CLAUDE.md as free text
- D. An environment variable with no config file

Correct answer: A. Shared, team-wide MCP servers belong in version-controlled .mcp.json; personal or experimental servers belong in the non-shared ~/.claude.json until they're ready to be proposed for the shared config.

Q45. Where should the configuration for a shared CI-status MCP server every contributor needs access to live?

- A. `~/.claude.json` (personal, not shared)
- B. Directly inside `CLAUDE.md` as free text
- C. An environment variable with no config file
- D. ✓ `.mcp.json` (project-level, version-controlled)

Correct answer: D. Shared, team-wide MCP servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/.claude.json` until they're ready to be proposed for the shared config.

Q46. Where should the configuration for a personal experimental MCP server for a side project, unrelated to the team repo live?

- A. Directly inside `CLAUDE.md` as free text
- B. An environment variable with no config file
- C. ✓ `~/.claude.json` (personal, not shared)
- D. `.mcp.json` (project-level, version-controlled)

Correct answer: C. Shared, team-wide MCP servers belong in version-controlled `.mcp.json`; personal or experimental servers belong in the non-shared `~/.claude.json` until they're ready to be proposed for the shared config.

Q47. A developer needs to supply an API token for a database connection MCP server inside `.mcp.json` without committing the raw secret. What's correct?

- A. Store the token as a comment in `CLAUDE.md`.
- B. ✓ Use environment-variable substitution, e.g. `${API_TOKEN}`, instead of a literal committed value.
- C. Commit the literal token directly in `.mcp.json` for simplicity.
- D. Base64-encode the token before committing it.

Correct answer: B. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q48. A developer needs to supply an API token for a payments-API MCP server inside `.mcp.json` without committing the raw secret. What's correct?

- A. ✓ Use environment-variable substitution, e.g. `${API_TOKEN}`, instead of a literal committed value.
- B. Commit the literal token directly in `.mcp.json` for simplicity.
- C. Base64-encode the token before committing it.
- D. Store the token as a comment in `CLAUDE.md`.

Correct answer: A. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q49. A developer needs to supply an API token for an internal ticketing MCP server inside `.mcp.json` without committing the raw secret. What's correct?

- A. Commit the literal token directly in `.mcp.json` for simplicity.
- B. Base64-encode the token before committing it.
- C. Store the token as a comment in `CLAUDE.md`.
- D. ✓ Use environment-variable substitution, e.g. `${API_TOKEN}`, instead of a literal committed value.

Correct answer: D. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q50. A developer needs to supply an API token for a cloud-storage MCP server inside `.mcp.json` without committing the raw secret. What's correct?

- A. Base64-encode the token before committing it.
- B. Store the token as a comment in `CLAUDE.md`.
- C. ✓ Use environment-variable substitution, e.g. `${API_TOKEN}`, instead of a literal committed value.
- D. Commit the literal token directly in `.mcp.json` for simplicity.

Correct answer: C. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q51. A developer needs to supply an API token for a CRM MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. Store the token as a comment in CLAUDE.md.
- B. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.
- C. Commit the literal token directly in .mcp.json for simplicity.
- D. Base64-encode the token before committing it.

Correct answer: B. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q52. A developer needs to supply an API token for an internal analytics MCP server inside .mcp.json without committing the raw secret. What's correct?

- A. ✓ Use environment-variable substitution, e.g. \${API_TOKEN}, instead of a literal committed value.
- B. Commit the literal token directly in .mcp.json for simplicity.
- C. Base64-encode the token before committing it.
- D. Store the token as a comment in CLAUDE.md.

Correct answer: A. Secrets should flow through environment-variable substitution rather than being committed as literal values — this keeps the shared, version-controlled config free of real credentials.

Q53. After Claude Code writes a large refactor, who should review it for issues?

- A. Have the same session that wrote the code also review it, to save time.
- B. Skip review entirely if the author is confident in the change.
- C. Have the same session review it twice in a row for extra assurance.
- D. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.

Correct answer: D. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q54. After Claude Code writes a security-sensitive authentication change, who should review it for issues?

- A. Skip review entirely if the author is confident in the change.
- B. Have the same session review it twice in a row for extra assurance.
- C. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- D. Have the same session that wrote the code also review it, to save time.

Correct answer: C. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q55. After Claude Code writes a database migration script, who should review it for issues?

- A. Have the same session review it twice in a row for extra assurance.
- B. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- C. Have the same session that wrote the code also review it, to save time.
- D. Skip review entirely if the author is confident in the change.

Correct answer: B. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q56. After Claude Code writes a payment-processing change, who should review it for issues?

- A. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- B. Have the same session that wrote the code also review it, to save time.
- C. Skip review entirely if the author is confident in the change.
- D. Have the same session review it twice in a row for extra assurance.

Correct answer: A. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q57. After Claude Code writes a public API contract change, who should review it for issues?

- A. Have the same session that wrote the code also review it, to save time.
- B. Skip review entirely if the author is confident in the change.
- C. Have the same session review it twice in a row for extra assurance.
- D. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.

Correct answer: D. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q58. After Claude Code writes a critical bug fix under time pressure, who should review it for issues?

- A. Skip review entirely if the author is confident in the change.
- B. Have the same session review it twice in a row for extra assurance.
- C. ✓ Have an independent session, with no generation context, perform the review — it's less likely to rationalize the original decisions.
- D. Have the same session that wrote the code also review it, to save time.

Correct answer: C. A session that just generated code carries the reasoning that produced it and tends to rationalize its own decisions in review; an independent session without that generation context catches more.

Q59. Which mechanism is the right fit for blocking any Bash command that contains "rm -rf"?

- A. A stronger system-prompt instruction
- B. ✓ A PreToolUse hook on the Bash tool
- C. A PostToolUse hook
- D. A note in CLAUDE.md

Correct answer: B. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q60. Which mechanism is the right fit for trimming an oversized tool result down to only the relevant fields before it reaches the model?

- A. ✓ A PostToolUse hook
- B. A PreToolUse hook
- C. A note in CLAUDE.md
- D. A stronger system-prompt instruction

Correct answer: A. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q61. Which mechanism is the right fit for blocking a refund tool call above a dollar threshold?

- A. A PostToolUse hook
- B. A note in CLAUDE.md
- C. A stronger system-prompt instruction
- D. ✓ A PreToolUse hook on the refund tool

Correct answer: D. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q62. Which mechanism is the right fit for logging every tool call's arguments for audit purposes?

- A. A note in CLAUDE.md
- B. A stronger system-prompt instruction
- C. ✓ A PreToolUse hook
- D. A PostToolUse hook

Correct answer: C. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q63. Which mechanism is the right fit for stripping sensitive fields out of a database query's result before it's shown to the model?

- A. A stronger system-prompt instruction
- B. ✓ A PostToolUse hook
- C. A PreToolUse hook
- D. A note in CLAUDE.md

Correct answer: B. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q64. Which mechanism is the right fit for preventing a deploy tool from running outside business hours?

- A. ✓ A PreToolUse hook on the deploy tool
- B. A PostToolUse hook
- C. A note in CLAUDE.md
- D. A stronger system-prompt instruction

Correct answer: A. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q65. Which mechanism is the right fit for recording how long each tool call took after it completes?

- A. A PreToolUse hook
- B. A note in CLAUDE.md
- C. A stronger system-prompt instruction
- D. ✓ A PostToolUse hook

Correct answer: D. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q66. Which mechanism is the right fit for requiring a confirmation flag be present before a delete tool executes?

- A. A note in CLAUDE.md
- B. A stronger system-prompt instruction
- C. ✓ A PreToolUse hook on the delete tool
- D. A PostToolUse hook

Correct answer: C. PreToolUse hooks intercept before a call executes (ideal for blocking/gating); PostToolUse hooks run after a call returns (ideal for trimming, redacting, or logging results).

Q67. For a documentation-writing session that never needs to run shell commands, what's the best practice around tool permissions?

- A. Leave permissions unscoped and review afterward.
- B. ✓ Restrict the session's allowed tools to only what the task requires.
- C. Grant every available tool by default for convenience.
- D. Rely solely on a prompt instruction not to use unnecessary tools.

Correct answer: B. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q68. For a read-only code-review session that shouldn't be able to edit files, what's the best practice around tool permissions?

- A. ✓ Restrict the session's allowed tools to only what the task requires.
- B. Grant every available tool by default for convenience.
- C. Rely solely on a prompt instruction not to use unnecessary tools.
- D. Leave permissions unscoped and review afterward.

Correct answer: A. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q69. For a session tasked only with running the test suite, not modifying source, what's the best practice around tool permissions?

- A. Grant every available tool by default for convenience.
- B. Rely solely on a prompt instruction not to use unnecessary tools.
- C. Leave permissions unscoped and review afterward.
- D. ✓ Restrict the session's allowed tools to only what the task requires.

Correct answer: D. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q70. For a session meant only to answer questions about the codebase, what's the best practice around tool permissions?

- A. Rely solely on a prompt instruction not to use unnecessary tools.
- B. Leave permissions unscoped and review afterward.
- C. ✓ Restrict the session's allowed tools to only what the task requires.
- D. Grant every available tool by default for convenience.

Correct answer: C. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q71. For a session doing a dependency audit that shouldn't be able to install packages, what's the best practice around tool permissions?

- A. Leave permissions unscoped and review afterward.
- B. ✓ Restrict the session's allowed tools to only what the task requires.
- C. Grant every available tool by default for convenience.
- D. Rely solely on a prompt instruction not to use unnecessary tools.

Correct answer: B. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q72. For a session drafting release notes that shouldn't touch the deploy pipeline, what's the best practice around tool permissions?

- A. ✓ Restrict the session's allowed tools to only what the task requires.
- B. Grant every available tool by default for convenience.
- C. Rely solely on a prompt instruction not to use unnecessary tools.
- D. Leave permissions unscoped and review afterward.

Correct answer: A. Scoping allowed tools to the minimum a task actually needs limits blast radius if the session goes off track — the same least-privilege principle applies inside Claude Code sessions as it does to agents generally.

Q73. In a monorepo where the /backend and /frontend packages need different linting conventions, what's the best CLAUDE.md structure?

- A. One single project-level CLAUDE.md trying to cover every package's rules at once
- B. Putting all conventions into each developer's personal user-level CLAUDE.md
- C. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- D. ✓ Separate directory-level CLAUDE.md files in each package

Correct answer: D. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q74. In a monorepo where the /mobile package needs its own build conventions distinct from the rest of the monorepo, what's the best CLAUDE.md structure?

- A. Putting all conventions into each developer's personal user-level CLAUDE.md
- B. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- C. ✓ A directory-level CLAUDE.md inside /mobile
- D. One single project-level CLAUDE.md trying to cover every package's rules at once

Correct answer: C. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q75. In a monorepo where the /infra package needs Terraform-specific conventions not relevant elsewhere, what's the best CLAUDE.md structure?

- A. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- B. ✓ A directory-level CLAUDE.md inside /infra
- C. One single project-level CLAUDE.md trying to cover every package's rules at once
- D. Putting all conventions into each developer's personal user-level CLAUDE.md

Correct answer: B. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q76. In a monorepo where the /shared package's conventions should apply to every consuming package, what's the best CLAUDE.md structure?

- A. ✓ A directory-level CLAUDE.md inside /shared, referenced or duplicated as needed
- B. One single project-level CLAUDE.md trying to cover every package's rules at once
- C. Putting all conventions into each developer's personal user-level CLAUDE.md
- D. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge

Correct answer: A. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q77. In a monorepo where the /legacy package needs different, more lenient conventions during a migration, what's the best CLAUDE.md structure?

- A. One single project-level CLAUDE.md trying to cover every package's rules at once
- B. Putting all conventions into each developer's personal user-level CLAUDE.md
- C. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- D. ✓ A directory-level CLAUDE.md inside /legacy

Correct answer: D. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q78. In a monorepo where the /docs package needs its own writing-style conventions distinct from code conventions, what's the best CLAUDE.md structure?

- A. Putting all conventions into each developer's personal user-level CLAUDE.md
- B. Skipping CLAUDE.md for sub-packages and relying on tribal knowledge
- C. ✓ A directory-level CLAUDE.md inside /docs
- D. One single project-level CLAUDE.md trying to cover every package's rules at once

Correct answer: C. Directory-level CLAUDE.md files let different parts of a monorepo carry their own conventions without cluttering a single project-level file with rules that only apply to one package.

Q79. During a Claude Code session, a session's context usage is climbing unexpectedly fast. What's the first thing to check?

- A. Ignore it, since context issues resolve themselves over time.
- B. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- C. Immediately switch to a larger context-window model with no further investigation.
- D. Restart the session repeatedly until the symptom happens to go away.

Correct answer: B. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q80. During a Claude Code session, a session seems to be repeating the same failed approach in a loop. What's the first thing to check?

- A. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- B. Immediately switch to a larger context-window model with no further investigation.
- C. Restart the session repeatedly until the symptom happens to go away.
- D. Ignore it, since context issues resolve themselves over time.

Correct answer: A. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q81. During a Claude Code session, token costs for a recurring automated task have crept up over time. What's the first thing to check?

- A. Immediately switch to a larger context-window model with no further investigation.
- B. Restart the session repeatedly until the symptom happens to go away.
- C. Ignore it, since context issues resolve themselves over time.
- D. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.

Correct answer: D. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q82. During a Claude Code session, a session appears to have lost track of an earlier instruction. What's the first thing to check?

- A. Restart the session repeatedly until the symptom happens to go away.
- B. Ignore it, since context issues resolve themselves over time.
- C. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- D. Immediately switch to a larger context-window model with no further investigation.

Correct answer: C. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q83. During a Claude Code session, a long session's responses have started degrading in quality. What's the first thing to check?

- A. Ignore it, since context issues resolve themselves over time.
- B. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- C. Immediately switch to a larger context-window model with no further investigation.
- D. Restart the session repeatedly until the symptom happens to go away.

Correct answer: B. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q84. During a Claude Code session, a CI job using Claude Code is taking far longer than expected. What's the first thing to check?

- A. ✓ Inspect what's actually being loaded into context (CLAUDE.md files, rules, tool outputs) and trim what isn't needed for the current task.
- B. Immediately switch to a larger context-window model with no further investigation.
- C. Restart the session repeatedly until the symptom happens to go away.
- D. Ignore it, since context issues resolve themselves over time.

Correct answer: A. Context bloat is usually diagnosable: check what's actually being loaded (broad CLAUDE.md files, unscoped rules, untrimmed tool output) before reaching for a bigger context window as a workaround.

Q85. A team frequently needs running the same multi-step release checklist every week. What's the most maintainable way to make that repeatable in Claude Code?

- A. Re-type the full multi-step prompt from memory every time it's needed.
- B. Paste the steps into the system prompt permanently regardless of task.
- C. Rely on the model to remember the steps from a prior unrelated session.
- D. ✓ Encode the repeatable steps as a custom slash command that encodes the checklist steps, invoked consistently instead of re-typed each time.

Correct answer: D. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q86. A team frequently needs generating a standardized PR description every time. What's the most maintainable way to make that repeatable in Claude Code?

- A. Paste the steps into the system prompt permanently regardless of task.
- B. Rely on the model to remember the steps from a prior unrelated session.
- C. ✓ Encode the repeatable steps as a custom slash command for PR descriptions, invoked consistently instead of re-typed each time.
- D. Re-type the full multi-step prompt from memory every time it's needed.

Correct answer: C. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q87. A team frequently needs running the same onboarding walkthrough for every new repo contributor. What's the most maintainable way to make that repeatable in Claude Code?

- A. Rely on the model to remember the steps from a prior unrelated session.
- B. ✓ Encode the repeatable steps as a custom slash command for onboarding, invoked consistently instead of re-typed each time.
- C. Re-type the full multi-step prompt from memory every time it's needed.
- D. Paste the steps into the system prompt permanently regardless of task.

Correct answer: B. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q88. A team frequently needs triggering the same dependency-audit routine periodically. What's the most maintainable way to make that repeatable in Claude Code?

- A. ✓ Encode the repeatable steps as a custom slash command for the audit routine, invoked consistently instead of re-typed each time.
- B. Re-type the full multi-step prompt from memory every time it's needed.
- C. Paste the steps into the system prompt permanently regardless of task.
- D. Rely on the model to remember the steps from a prior unrelated session.

Correct answer: A. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q89. A team frequently needs producing a consistent weekly status report format. What's the most maintainable way to make that repeatable in Claude Code?

- A. Re-type the full multi-step prompt from memory every time it's needed.
- B. Paste the steps into the system prompt permanently regardless of task.
- C. Rely on the model to remember the steps from a prior unrelated session.
- D. ✓ Encode the repeatable steps as a custom slash command for the status report, invoked consistently instead of re-typed each time.

Correct answer: D. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q90. A team frequently needs running the same pre-release smoke-test sequence. What's the most maintainable way to make that repeatable in Claude Code?

- A. Paste the steps into the system prompt permanently regardless of task.
- B. Rely on the model to remember the steps from a prior unrelated session.
- C. ✓ Encode the repeatable steps as a custom slash command for the smoke-test sequence, invoked consistently instead of re-typed each time.
- D. Re-type the full multi-step prompt from memory every time it's needed.

Correct answer: C. Repeatable, well-defined workflows are good candidates for custom slash commands — consistent, version-controllable, and easier to maintain than re-typing the same steps.

Q91. For a quick, low-stakes formatting fix, how should the thinking/reasoning budget typically be set?

- A. Pick a thinking budget at random for each task.
- B. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- C. Always use the maximum thinking budget regardless of task complexity.
- D. Always disable extended thinking to save tokens, regardless of task complexity.

Correct answer: B. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q92. For a complex multi-file architectural refactor, how should the thinking/reasoning budget typically be set?

- A. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- B. Always use the maximum thinking budget regardless of task complexity.
- C. Always disable extended thinking to save tokens, regardless of task complexity.
- D. Pick a thinking budget at random for each task.

Correct answer: A. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q93. For a one-line typo correction, how should the thinking/reasoning budget typically be set?

- A. Always use the maximum thinking budget regardless of task complexity.
- B. Always disable extended thinking to save tokens, regardless of task complexity.
- C. Pick a thinking budget at random for each task.
- D. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.

Correct answer: D. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q94. For a subtle concurrency bug requiring deep analysis, how should the thinking/reasoning budget typically be set?

- A. Always disable extended thinking to save tokens, regardless of task complexity.
- B. Pick a thinking budget at random for each task.
- C. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- D. Always use the maximum thinking budget regardless of task complexity.

Correct answer: C. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q95. For a routine dependency version bump, how should the thinking/reasoning budget typically be set?

- A. Pick a thinking budget at random for each task.
- B. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- C. Always use the maximum thinking budget regardless of task complexity.
- D. Always disable extended thinking to save tokens, regardless of task complexity.

Correct answer: B. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q96. For a tricky algorithmic optimization problem, how should the thinking/reasoning budget typically be set?

- A. ✓ Scale the reasoning/thinking budget to the task's actual difficulty rather than using one fixed setting for everything.
- B. Always use the maximum thinking budget regardless of task complexity.
- C. Always disable extended thinking to save tokens, regardless of task complexity.
- D. Pick a thinking budget at random for each task.

Correct answer: A. Matching reasoning depth to task complexity avoids both under-thinking genuinely hard problems and wasting tokens over-thinking trivial ones.

Q97. For a terse internal script meant to be piped into another tool, what output style best fits the consumer of the response?

- A. A single fixed output style used for every task regardless of audience
- B. Whatever the model defaults to, with no configuration
- C. The most verbose style available, regardless of the consumer
- D. ✓ A minimal, non-conversational output style

Correct answer: D. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q98. For a long-form explanatory document meant for a non-technical stakeholder, what output style best fits the consumer of the response?

- A. Whatever the model defaults to, with no configuration
- B. The most verbose style available, regardless of the consumer
- C. ✓ A more verbose, explanatory output style
- D. A single fixed output style used for every task regardless of audience

Correct answer: C. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q99. For a CI log parser expecting machine-readable output only, what output style best fits the consumer of the response?

- A. The most verbose style available, regardless of the consumer
- B. ✓ A minimal, non-conversational output style
- C. A single fixed output style used for every task regardless of audience
- D. Whatever the model defaults to, with no configuration

Correct answer: B. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q100. For an onboarding guide meant to teach a new engineer the codebase, what output style best fits the consumer of the response?

- A. ✓ A more verbose, explanatory output style
- B. A single fixed output style used for every task regardless of audience
- C. Whatever the model defaults to, with no configuration
- D. The most verbose style available, regardless of the consumer

Correct answer: A. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q101. For a one-line status check consumed by another automated script, what output style best fits the consumer of the response?

- A. A single fixed output style used for every task regardless of audience
- B. Whatever the model defaults to, with no configuration
- C. The most verbose style available, regardless of the consumer
- D. ✓ A minimal, non-conversational output style

Correct answer: D. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q102. For a detailed design-review writeup meant for a cross-functional audience, what output style best fits the consumer of the response?

- A. Whatever the model defaults to, with no configuration
- B. The most verbose style available, regardless of the consumer
- C. ✓ A more verbose, explanatory output style
- D. A single fixed output style used for every task regardless of audience

Correct answer: C. Output style should match who or what is consuming the result — machine-readable consumers need minimal, parseable output; human readers benefit from more explanatory framing.

Q103. When integrating Claude Code into a git workflow, is committing directly to the main branch on every change good practice?

- A. It's fine as long as the change was AI-generated.
- B. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- C. This is standard best practice and should be encouraged.
- D. It only matters for large teams, not small ones.

Correct answer: B. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q104. When integrating Claude Code into a git workflow, is generating a commit message with no reference to the actual diff good practice?

- A. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- B. This is standard best practice and should be encouraged.
- C. It only matters for large teams, not small ones.
- D. It's fine as long as the change was AI-generated.

Correct answer: A. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q105. When integrating Claude Code into a git workflow, is opening a PR with no description of what changed or why good practice?

- A. This is standard best practice and should be encouraged.
- B. It only matters for large teams, not small ones.
- C. It's fine as long as the change was AI-generated.
- D. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.

Correct answer: D. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q106. When integrating Claude Code into a git workflow, is force-pushing over a shared branch without warning good practice?

- A. It only matters for large teams, not small ones.
- B. It's fine as long as the change was AI-generated.
- C. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- D. This is standard best practice and should be encouraged.

Correct answer: C. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q107. When integrating Claude Code into a git workflow, is committing generated code without running the test suite first good practice?

- A. It's fine as long as the change was AI-generated.
- B. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- C. This is standard best practice and should be encouraged.
- D. It only matters for large teams, not small ones.

Correct answer: B. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q108. When integrating Claude Code into a git workflow, is merging without any review step for a sensitive change good practice?

- A. ✓ Avoid it — use feature branches, diff-grounded commit messages, descriptive PRs, and a review/test gate before merge.
- B. This is standard best practice and should be encouraged.
- C. It only matters for large teams, not small ones.
- D. It's fine as long as the change was AI-generated.

Correct answer: A. Claude Code should be integrated into the same disciplined git practices as human contributors: branches, meaningful commit messages, descriptive PRs, and a review/test gate before merge.

Q109. On a growing team, a new hire doesn't know which conventions apply to which part of a large codebase. What's the durable fix?

- A. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- B. Rely on the new person to ask the right questions unprompted.
- C. Leave conventions undocumented since Claude Code should infer them automatically.
- D. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.

Correct answer: D. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q110. On a growing team, a new contractor keeps re-deriving conventions the team already settled on months ago. What's the durable fix?

- A. Rely on the new person to ask the right questions unprompted.
- B. Leave conventions undocumented since Claude Code should infer them automatically.
- C. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- D. Have the new person shadow a senior engineer indefinitely instead of documenting anything.

Correct answer: C. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q111. On a growing team, a new team member's Claude Code sessions don't reflect the team's actual coding standards. What's the durable fix?

- A. Leave conventions undocumented since Claude Code should infer them automatically.
- B. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- C. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- D. Rely on the new person to ask the right questions unprompted.

Correct answer: B. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q112. On a growing team, an open-source contributor's PRs don't follow the project's established patterns.

What's the durable fix?

- A. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- B. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- C. Rely on the new person to ask the right questions unprompted.
- D. Leave conventions undocumented since Claude Code should infer them automatically.

Correct answer: A. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q113. On a growing team, a new hire wastes time discovering tribal knowledge that was never written down.

What's the durable fix?

- A. Have the new person shadow a senior engineer indefinitely instead of documenting anything.
- B. Rely on the new person to ask the right questions unprompted.
- C. Leave conventions undocumented since Claude Code should infer them automatically.
- D. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.

Correct answer: D. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q114. On a growing team, a rotating on-call engineer isn't aware of the project's incident-response conventions.

What's the durable fix?

- A. Rely on the new person to ask the right questions unprompted.
- B. Leave conventions undocumented since Claude Code should infer them automatically.
- C. ✓ Ensure the relevant conventions live in version-controlled project- or directory-level CLAUDE.md files, not in any one person's head or personal config.
- D. Have the new person shadow a senior engineer indefinitely instead of documenting anything.

Correct answer: C. Project- and directory-level CLAUDE.md files, being shared and version-controlled, are exactly the mechanism meant to make conventions available to every contributor from day one — not personal config or informal habit.

Q115. A team wants to guarantee that unit tests should use a specific mocking library, not whatever the model defaults to. What's the most reliable way to encode this for Claude Code?

- A. Leave it as informal team knowledge with no configuration.
- B. ✓ Encode it as a path-scoped rule in .claude/rules/ (or project CLAUDE.md) so it loads automatically whenever test files are touched.
- C. Mention it once in a Slack message and hope it's remembered.
- D. Rely on code review alone to catch every violation after the fact.

Correct answer: B. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q116. A team wants to guarantee that integration tests must run against a seeded test database, never production. What's the most reliable way to encode this for Claude Code?

- A. ✓ Encode it as a path-scoped rule in .claude/rules/ (or project CLAUDE.md) so it loads automatically whenever test files are touched.
- B. Mention it once in a Slack message and hope it's remembered.
- C. Rely on code review alone to catch every violation after the fact.
- D. Leave it as informal team knowledge with no configuration.

Correct answer: A. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q117. A team wants to guarantee that test files must follow a specific naming and directory pattern. What's the most reliable way to encode this for Claude Code?

- A. Mention it once in a Slack message and hope it's remembered.
- B. Rely on code review alone to catch every violation after the fact.
- C. Leave it as informal team knowledge with no configuration.
- D. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.

Correct answer: D. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q118. A team wants to guarantee that snapshot tests require explicit review before being accepted. What's the most reliable way to encode this for Claude Code?

- A. Rely on code review alone to catch every violation after the fact.
- B. Leave it as informal team knowledge with no configuration.
- C. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.
- D. Mention it once in a Slack message and hope it's remembered.

Correct answer: C. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q119. A team wants to guarantee that flaky tests must be quarantined rather than silently skipped. What's the most reliable way to encode this for Claude Code?

- A. Leave it as informal team knowledge with no configuration.
- B. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.
- C. Mention it once in a Slack message and hope it's remembered.
- D. Rely on code review alone to catch every violation after the fact.

Correct answer: B. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

Q120. A team wants to guarantee that coverage thresholds must be checked before a PR is considered mergeable. What's the most reliable way to encode this for Claude Code?

- A. ✓ Encode it as a path-scoped rule in `.claude/rules/` (or project `CLAUDE.md`) so it loads automatically whenever test files are touched.
- B. Mention it once in a Slack message and hope it's remembered.
- C. Rely on code review alone to catch every violation after the fact.
- D. Leave it as informal team knowledge with no configuration.

Correct answer: A. Path-scoped rules (or project-level CLAUDE.md) make a convention load automatically and consistently whenever relevant files are touched, rather than depending on memory or after-the-fact review.

End of Domain 2 deep-dive. This is an independent study aid; verify current exam logistics and blueprint details against your Claude Partner Network portal before sitting the exam.